

▼ Day 25: GPU FinOps & Cost Optimization - Hands-on Lab

This notebook connects to a local Docker Compose cluster (via ngrok/cloudflared tunnel) to demonstrate:

1. **GPU Cluster Monitoring** - Real-time GPU metrics
2. **Cost Tracking (OpenCost-like)** - Per-node cost allocation & waste analysis
3. **Spot Instance Management** - Bidding, preemption, savings
4. **Autoscaling (KEDA-like)** - Policy-based GPU scaling
5. **Budget Management & Forecasting** - Alerts, burn rate

Prerequisites

- Docker Compose running locally with the GPU FinOps Lab services
- Tunnel exposed via ngrok or cloudflared (free tier)

```
1 # Cell 1: Setup
2 !pip install -q requests pandas matplotlib plotly
3
4 import requests
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import time
8 import json
9 from IPython.display import display, HTML, clear_output
```

```
1 # Cell 2: Configure Gateway URL
2 # Replace with your ngrok/cloudflared URL after running the tunnel
3 GATEWAY_URL = "https://foam-academy-excessive-cure.trycloudflare.com/" # <-- CHANGE THIS
4
5 # Test connection
6 try:
7     r = requests.get(f"{GATEWAY_URL}/", timeout=10)
8     print("✅ Connected to GPU FinOps Lab Gateway")
9     print(json.dumps(r.json(), indent=2))
10 except Exception as e:
11     print(f"❌ Connection failed: {e}")
12     print("Make sure Docker Compose is running and tunnel is active")
```

```
✅ Connected to GPU FinOps Lab Gateway
{
  "service": "GPU FinOps Lab Gateway",
  "endpoints": {
    "cluster": "/cluster/*",
    "billing": "/billing/*",
    "spot": "/spot/*",
    "autoscaler": "/autoscaler/*",
    "cost": "/cost/*"
  }
}
```

▼ Part 1: GPU Cluster Monitoring

Observe the mock GPU cluster state — nodes, utilization, memory, power draw.

```
1 # Cell 3: View Cluster Nodes
2 nodes = requests.get(f"{GATEWAY_URL}/cluster/nodes").json()
3
4 print(f"🖥️ Cluster has {len(nodes)} nodes")
5 print("=" * 60)
6
7 for node_id, gpus in nodes.items():
8     print(f"\n💻 {node_id}:")
9     for gpu in gpus:
10         status_icon = "🟢" if gpu['status'] == 'idle' else "🔴"
11         print(f"    GPU {gpu['gpu_id']} [{gpu['gpu_type']}] {status_icon} "
12               f"Util: {gpu['utilization']:.1f}% | "
13               f"Mem: {gpu['memory_used_gb']:.1f}/{gpu['memory_total_gb']}GB | ")
```

```
14 f"Power: {gpu['power_draw_watts']:.0f}W | "  
15 f"Temp: {gpu['temperature_c']:.0f}°C")
```

Cluster has 8 nodes

node-00:

GPU 0 [T4] Util: 61.4% | Mem: 10.5/16.0GB | Power: 46W | Temp: 69°C

GPU 1 [T4] Util: 87.7% | Mem: 13.9/16.0GB | Power: 44W | Temp: 72°C

node-01:

GPU 0 [A100] Util: 89.8% | Mem: 65.1/80.0GB | Power: 187W | Temp: 66°C

GPU 1 [A100] Util: 84.3% | Mem: 40.2/80.0GB | Power: 231W | Temp: 77°C

node-02:

GPU 0 [V100] Util: 85.6% | Mem: 19.4/32.0GB | Power: 198W | Temp: 70°C

GPU 1 [V100] Util: 9.8% | Mem: 0.6/32.0GB | Power: 28W | Temp: 36°C

node-03:

GPU 0 [T4] Util: 88.9% | Mem: 13.4/16.0GB | Power: 42W | Temp: 63°C

GPU 1 [T4] Util: 73.8% | Mem: 13.6/16.0GB | Power: 65W | Temp: 70°C

node-04:

GPU 0 [T4] Util: 82.1% | Mem: 10.4/16.0GB | Power: 47W | Temp: 76°C

GPU 1 [T4] Util: 83.3% | Mem: 12.5/16.0GB | Power: 50W | Temp: 67°C

node-05:

GPU 0 [T4] Util: 86.3% | Mem: 8.5/16.0GB | Power: 49W | Temp: 59°C

GPU 1 [T4] Util: 67.6% | Mem: 12.5/16.0GB | Power: 59W | Temp: 82°C

node-06:

GPU 0 [T4] Util: 7.1% | Mem: 1.6/16.0GB | Power: 24W | Temp: 42°C

GPU 1 [T4] Util: 4.3% | Mem: 1.9/16.0GB | Power: 25W | Temp: 31°C

node-07:

GPU 0 [T4] Util: 0.0% | Mem: 0.5/16.0GB | Power: 20W | Temp: 30°C

GPU 1 [T4] Util: 0.0% | Mem: 0.5/16.0GB | Power: 20W | Temp: 30°C

```
1 # Cell 4: Cluster Metrics Summary  
2 metrics = requests.get(f"{GATEWAY_URL}/cluster/metrics").json()  
3  
4 print("Cluster Metrics")  
5 print("=" * 40)  
6 print(f"Total GPUs:           {metrics['total_gpus']}")  
7 print(f"Busy GPUs:                {metrics['busy_gpus']}")  
8 print(f"Idle GPUs:                 {metrics['idle_gpus']}")  
9 print(f"Avg Utilization:         {metrics['avg_utilization']:.1f}%")  
10 print(f"Memory Used:               {metrics['total_memory_used_gb']:.1f} GB")  
11 print(f"Memory Capacity:          {metrics['total_memory_capacity_gb']:.1f} GB")  
12 print(f"Total Power Draw:         {metrics['total_power_draw_watts']:.0f} W")  
13 print(f"Node Count:               {metrics['node_count']}")
```

Cluster Metrics

Total GPUs: 16

Busy GPUs: 11

Idle GPUs: 5

Avg Utilization: 57.0%

Memory Used: 225.0 GB

Memory Capacity: 416.0 GB

Total Power Draw: 1136 W

Node Count: 8

Part 2: Workload Submission & Cost Tracking

Submit GPU workloads and track their costs in real-time.

```
1 # Cell 5: Submit multiple workloads  
2 workloads = [  
3     {"workload_id": "train-resnet-001", "gpu_type_preferred": "T4", "gpu_count": 1, "duration_seconds": 300},  
4     {"workload_id": "train-bert-002", "gpu_type_preferred": "A100", "gpu_count": 1, "duration_seconds": 600},  
5     {"workload_id": "inference-api-003", "gpu_type_preferred": "T4", "gpu_count": 1, "duration_seconds": 120},  
6     {"workload_id": "train-llm-004", "gpu_type_preferred": "A100", "gpu_count": 2, "duration_seconds": 900},  
7 ]  
8  
9 print("Submitting workloads...")  
10 for wl in workloads:
```

```
11 r = requests.post(f"{GATEWAY_URL}/cluster/workloads/submit", json=w1)
12 result = r.json()
13 print(f"    w1['workload_id']: {result['status']} → {result.get('assigned', 'queued')}")
14
15 # Check cluster state after submission
16 print("\n📊 Updated metrics:")
17 metrics = requests.get(f"{GATEWAY_URL}/cluster/metrics").json()
18 print(f"    Busy GPUs: {metrics['busy_gpus']}/{metrics['total_gpus']} | Utilization: {metrics['avg_utilization']:.1f}%")
```

🚀 Submitting workloads...

train-resnet-001: running → [{'node_id': 'node-06', 'gpu_id': 0}]
train-bert-002: running → [{'node_id': 'node-02', 'gpu_id': 1}]
inference-api-003: running → [{'node_id': 'node-06', 'gpu_id': 1}]
train-llm-004: running → [{'node_id': 'node-07', 'gpu_id': 0}, {'node_id': 'node-07', 'gpu_id': 1}]

📊 Updated metrics:
Busy GPUs: 16/16 | Utilization: 80.7%

```
1 # Cell 6: Record billing for workloads
2 billing_events = [
3     {"workload_id": "train-resnet-001", "gpu_type": "T4", "gpu_count": 1, "duration_seconds": 300, "is_spot": False},
4     {"workload_id": "train-bert-002", "gpu_type": "A100", "gpu_count": 1, "duration_seconds": 600, "is_spot": False},
5     {"workload_id": "inference-api-003", "gpu_type": "T4", "gpu_count": 1, "duration_seconds": 120, "is_spot": True},
6     {"workload_id": "train-llm-004", "gpu_type": "A100", "gpu_count": 2, "duration_seconds": 900, "is_spot": True},
7 ]
8
9 print("🔥 Recording billing events...")
10 for event in billing_events:
11     r = requests.post(f"{GATEWAY_URL}/billing/record", json=event)
12     result = r.json()
13     spot_label = "[SPOT]" if event['is_spot'] else "[ON-DEMAND]"
14     print(f"    {event['workload_id']} {spot_label}: ${result['total_cost_usd']:.4f} "
15           f"(saved ${result['savings_usd']:.4f})")
16
17 # Get billing summary
18 print("\n📊 Billing Summary:")
19 summary = requests.get(f"{GATEWAY_URL}/billing/summary").json()
20 print(f"    Total Cost:    ${summary['total_cost_usd']:.4f}")
21 print(f"    Total Savings:  ${summary['total_savings_usd']:.4f}")
22 print(f"    Budget Used:    {summary['budget_utilization_pct']:.1f}%")
23 print(f"    Alert Status:   {summary['alert']}")
```

🔥 Recording billing events...

train-resnet-001 [ON-DEMAND]: \$0.0292 (saved \$0.0000)
train-bert-002 [ON-DEMAND]: \$0.6117 (saved \$0.0000)
inference-api-003 [SPOT]: \$0.0035 (saved \$0.0082)
train-llm-004 [SPOT]: \$0.5505 (saved \$1.2845)

📊 Billing Summary:
Total Cost: \$5.2869
Total Savings: \$5.5380
Budget Used: 5.3%
Alert Status: OK

▼ Part 3: Spot Instance Management

Demonstrate spot bidding, preemption handling, and cost savings.

```
1 # Cell 7: Check spot pricing
2 spot_pricing = requests.get(f"{GATEWAY_URL}/spot/pricing").json()
3
4 print("✅ Current Spot Pricing")
5 print("=" * 60)
6 print(f"{'GPU Type':<10} {'On-Demand':<12} {'Spot Price':<12} {'Discount':<10} {'Availability'}")
7 print("-" * 60)
8 for gpu, info in spot_pricing.items():
9     print(f"{'gpu':<10} ${info['on_demand_price']:<10.2f} ${info['current_spot_price']:<10.4f} "
10           f"{'info['discount_pct']:<8.1f}% {'info['availability']}]")
```

✅ Current Spot Pricing

GPU Type	On-Demand	Spot Price	Discount	Availability
T4	\$0.35	\$0.2371	32.3	% high

A100	\$3.67	\$2.2409	38.9	% high
V100	\$2.48	\$1.3977	43.6	% low

```

1 # Cell 8: Request spot instances
2 spot_requests = [
3     {"instance_id": "spot-t4-001", "gpu_type": "T4", "gpu_count": 1, "max_price_per_hour": 0.15, "workload_id": "batch-job-
4     {"instance_id": "spot-t4-002", "gpu_type": "T4", "gpu_count": 1, "max_price_per_hour": 0.12, "workload_id": "batch-job-
5     {"instance_id": "spot-a100-001", "gpu_type": "A100", "gpu_count": 1, "max_price_per_hour": 1.50, "workload_id": "traini
6 ]
7
8 print("🔌 Requesting Spot Instances...")
9 for req in spot_requests:
10     r = requests.post(f"{GATEWAY_URL}/spot/request", json=req)
11     result = r.json()
12     status_icon = "✅" if result['status'] == 'granted' else "❌"
13     print(f"    {status_icon} {req['instance_id']} ({req['gpu_type']}): {result['status']}")
14     if result['status'] == 'rejected':
15         print(f"        Reason: {result.get('reason', 'N/A')}")

```

🔌 Requesting Spot Instances...

- ✅ spot-t4-001 (T4): granted
- ✅ spot-t4-002 (T4): granted
- ✅ spot-a100-001 (A100): granted

```

1 # Cell 9: Simulate spot preemption
2 print("⚡ Simulating spot preemption event...")
3 preemption = requests.post(f"{GATEWAY_URL}/spot/simulate-preemption").json()
4
5 print(f"\n   Preempted instances: {preemption['preempted_count']}")
6 print(f"   Still active: {preemption['total_active']}")
7
8 if preemption['preempted_instances']:
9     print("\n   ⚠️ Preempted:")
10    for p in preemption['preempted_instances']:
11        print(f"       - {p['instance_id']} (ran for {p['runtime_seconds']:.0f}s, "
12              f"       {p['notice_seconds']}s warning)")
13
14 # Savings report
15 print("\n💰 Spot Savings Report:")
16 savings = requests.get(f"{GATEWAY_URL}/spot/savings-report").json()
17 print(f"   Spot cost:      ${savings['total_spot_cost']:.4f}")
18 print(f"   On-demand equiv:  ${savings['on_demand_equivalent']:.4f}")
19 print(f"   Total saved:     ${savings['total_savings']:.4f} ({savings['savings_pct']:.1f}%)")

```

⚡ Simulating spot preemption event...

Preempted instances: 1
Still active: 5

⚠️ Preempted:
- opt-spot-1 (ran for 40s, 120s warning)

💰 Spot Savings Report:
Spot cost: \$0.0026
On-demand equiv: \$0.0087
Total saved: \$0.0061 (70.0%)

▼ Part 4: Autoscaling (KEDA-like)

Configure and trigger GPU autoscaling based on utilization thresholds.

```

1 # Cell 10: View and update autoscaling policy
2 policy = requests.get(f"{GATEWAY_URL}/autoscaler/policy").json()
3 print("📄 Current Autoscaling Policy:")
4 for k, v in policy.items():
5     print(f"    {k}: {v}")
6
7 # Update policy - more aggressive scaling
8 new_policy = {
9     "scale_up_threshold": 70,
10    "scale_down_threshold": 25,
11    "cooldown_seconds": 30,
12    "max_nodes": 10,
13    "min_nodes": 2,

```

```

14     "preferred_gpu_type": "T4",
15     "cost_aware": True
16 }
17
18 print("\n🔄 Updating policy...")
19 updated = requests.post(f"{GATEWAY_URL}/autoscaler/policy", json=new_policy).json()
20 print("✅ Policy updated")

```

📄 Current Autoscaling Policy:

```

scale_up_threshold: 70.0
scale_down_threshold: 25.0
cooldown_seconds: 30
max_nodes: 10
min_nodes: 2
preferred_gpu_type: T4
cost_aware: True

```

🔄 Updating policy...
✅ Policy updated

```

1 # Cell 11: Trigger autoscaler evaluation
2 print("🔄 Evaluating autoscaling decision...")
3 decision = requests.post(f"{GATEWAY_URL}/autoscaler/evaluate").json()
4
5 action_icons = {"scale_up": "⬆️", "scale_down": "⬇️", "no_action": "⏸️"}
6 print(f"\n   {action_icons.get(decision['action'], '?')} Action: {decision['action'].upper()}")
7 print(f"   Reason: {decision['reason']}")
8 print(f"   Current utilization: {decision['current_utilization']:.1f}%")
9 print(f"   Nodes: {decision['node_count']} → {decision['target_node_count']}")
10
11 # Run multiple evaluations to show scaling behavior
12 print("\n🔄 Running 5 evaluation cycles...")
13 for i in range(5):
14     time.sleep(2)
15     d = requests.post(f"{GATEWAY_URL}/autoscaler/evaluate").json()
16     print(f"   Cycle {i+1}: {d['action']}<12} | Util: {d['current_utilization']:.1f}% | Nodes: {d['node_count']}→{d['target

```

🔄 Evaluating autoscaling decision...

⬆️ Action: SCALE_UP
Reason: Utilization 80.7% > threshold 70.0%
Current utilization: 80.7%
Nodes: 8 → 9

🔄 Running 5 evaluation cycles...

Cycle 1:	no_action	Util: 71.8%	Nodes: 9→9
Cycle 2:	no_action	Util: 71.8%	Nodes: 9→9
Cycle 3:	no_action	Util: 71.8%	Nodes: 9→9
Cycle 4:	no_action	Util: 71.8%	Nodes: 9→9
Cycle 5:	no_action	Util: 71.8%	Nodes: 9→9

✓ Part 5: Cost Analysis & Optimization (OpenCost-like)

Analyze cost allocation, identify waste, and get recommendations.

```

1 # Cell 12: Take cost snapshots
2 print("📄 Taking cost snapshots...")
3 snapshots = []
4 for i in range(5):
5     snap = requests.post(f"{GATEWAY_URL}/cost/snapshot").json()
6     snapshots.append(snap)
7     print(f"   Snapshot {i+1}: Total=${snap['total_cost_usd']:.6f} | "
8           f"Idle=${snap['total_idle_cost_usd']:.6f} | "
9           f"Waste={snap['waste_pct']:.1f}%")
10    time.sleep(1)

```

📄 Taking cost snapshots...

Snapshot 1:	Total=\$0.047778	Idle=\$0.001944	Waste=4.1%
Snapshot 2:	Total=\$0.047778	Idle=\$0.001944	Waste=4.1%
Snapshot 3:	Total=\$0.047778	Idle=\$0.001944	Waste=4.1%
Snapshot 4:	Total=\$0.047778	Idle=\$0.001944	Waste=4.1%
Snapshot 5:	Total=\$0.047778	Idle=\$0.001944	Waste=4.1%

```

1 # Cell 13: Waste Report
2 waste = requests.get(f"{GATEWAY_URL}/cost/waste-report").json()

```

```

3
4 print("🗑️ WASTE ANALYSIS REPORT")
5 print("=" * 50)
6 print(f"    Average Waste:           {waste.get('avg_waste_pct', 0):.1f}%")
7 print(f"    Total Idle Cost:          ${waste.get('total_idle_cost_usd', 0):.6f}")
8 print(f"    Total Cost:               ${waste.get('total_cost_usd', 0):.6f}")
9 print(f"    Potential Monthly Save:   ${waste.get('potential_monthly_savings', 0):.2f}")
10 print(f"    Severity:                 {waste.get('severity', 'N/A')}")

```

```

🗑️ WASTE ANALYSIS REPORT
=====
Average Waste:           4.2%
Total Idle Cost:          $0.019440
Total Cost:               $0.460279
Potential Monthly Save:   $503.88
Severity:                 LOW

```

```

1 # Cell 14: Get Optimization Recommendations
2 recs = requests.post(f"{GATEWAY_URL}/cost/recommendations").json()
3
4 print("💡 COST OPTIMIZATION RECOMMENDATIONS")
5 print("=" * 60)
6 for i, rec in enumerate(recs, 1):
7     priority_icons = {"HIGH": "🔴", "MEDIUM": "🟡", "LOW": "🟢"}
8     icon = priority_icons.get(rec['priority'], '🟡')
9     print(f"\n{i}. {icon} [{rec['priority']}] {rec['description']}")
10    print(f"    Estimated savings: {rec['estimated_savings_pct']:.1f}%")

```

```

💡 COST OPTIMIZATION RECOMMENDATIONS
=====
1. 🟡 [MEDIUM] USE_SPOT
   Switch fault-tolerant workloads to spot instances for 60-70% savings.
   Estimated savings: 65.0%

2. 🟢 [LOW] SCHEDULING
   Schedule non-urgent training jobs during off-peak hours for lower spot prices.
   Estimated savings: 20.0%

```

```

1 # Cell 15: Full Dashboard View
2 dashboard = requests.get(f"{GATEWAY_URL}/cost/dashboard").json()
3
4 print("📊 GPU FinOps DASHBOARD")
5 print("=" * 60)
6
7 cm = dashboard.get('cluster_metrics', {})
8 print(f"\n🖥️ CLUSTER: {cm.get('total_gpus', 0)} GPUs across {cm.get('node_count', 0)} nodes")
9 print(f"    Utilization: {cm.get('avg_utilization', 0):.1f}% | Busy: {cm.get('busy_gpus', 0)} | Idle: {cm.get('idle_gpus', 0)}")
10
11 bs = dashboard.get('billing_summary', {})
12 print(f"\n💰 BILLING: ${bs.get('total_cost_usd', 0):.4f} / ${bs.get('budget_usd', 0):.2f} budget")
13 print(f"    Alert: {bs.get('alert', 'N/A')} | Savings: ${bs.get('total_savings_usd', 0):.4f}")
14
15 ss = dashboard.get('spot_savings', {})
16 print(f"\n📈 SPOT: Saved ${ss.get('total_savings', 0):.4f} ({ss.get('savings_pct', 0):.1f}%)")
17
18 wa = dashboard.get('waste_analysis', {})
19 if wa:
20     print(f"\n🗑️ WASTE: {wa.get('avg_waste_pct', 0):.1f}% | Severity: {wa.get('severity', 'N/A')}")

```

```

📊 GPU FinOps DASHBOARD
=====

🖥️ CLUSTER: 18 GPUs across 9 nodes
   Utilization: 71.8% | Busy: 16 | Idle: 2

💰 BILLING: $5.2869 / $100.00 budget
   Alert: OK | Savings: $5.5380

📈 SPOT: Saved $0.0253 (70.0%)

🗑️ WASTE: 4.2% | Severity: LOW

```

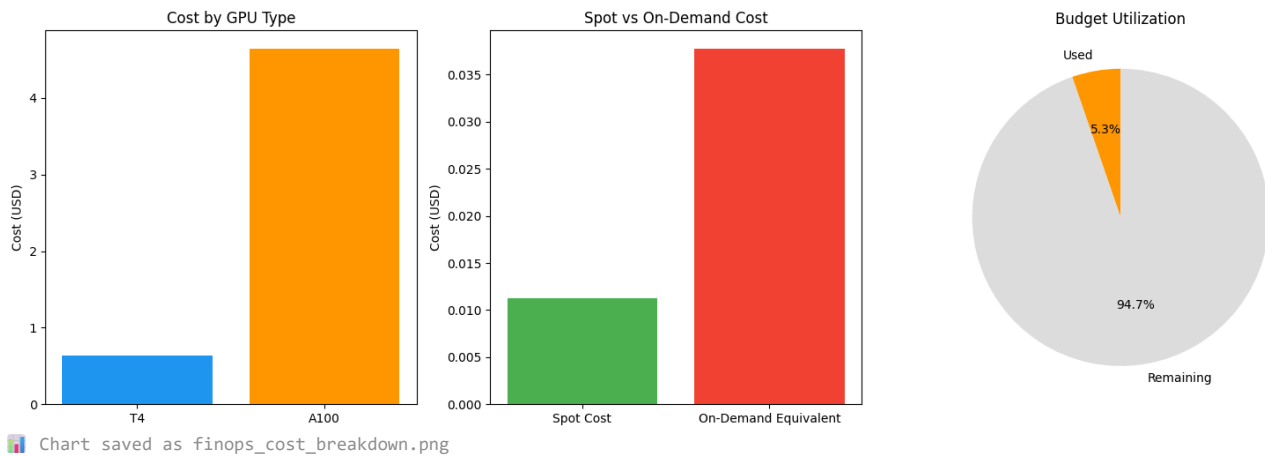
Part 6: Visualization

Create charts for cost breakdown and utilization trends.

```

1 # Cell 16: Cost Breakdown Visualization
2 summary = requests.get(f"{GATEWAY_URL}/billing/summary").json()
3
4 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
5
6 # Chart 1: Cost by GPU type
7 cost_by_type = summary.get('cost_by_gpu_type', {})
8 if cost_by_type:
9     types = list(cost_by_type.keys())
10    costs = [cost_by_type[t]['cost'] for t in types]
11    axes[0].bar(types, costs, color=['#2196F3', '#FF9800', '#4CAF50'])
12    axes[0].set_title('Cost by GPU Type')
13    axes[0].set_ylabel('Cost (USD)')
14
15 # Chart 2: Spot vs On-Demand
16 spot_data = requests.get(f"{GATEWAY_URL}/spot/savings-report").json()
17 labels = ['Spot Cost', 'On-Demand Equivalent']
18 values = [spot_data.get('total_spot_cost', 0), spot_data.get('on_demand_equivalent', 0)]
19 axes[1].bar(labels, values, color=['#4CAF50', '#F44336'])
20 axes[1].set_title('Spot vs On-Demand Cost')
21 axes[1].set_ylabel('Cost (USD)')
22
23 # Chart 3: Budget utilization
24 budget_used = summary.get('budget_utilization_pct', 0)
25 axes[2].pie([budget_used, 100-budget_used], labels=['Used', 'Remaining'],
26             colors=['#FF9800', '#E0E0E0'], autopct='%1.1f%%', startangle=90)
27 axes[2].set_title('Budget Utilization')
28
29 plt.tight_layout()
30 plt.savefig('finops_cost_breakdown.png', dpi=150, bbox_inches='tight')
31 plt.show()
32 print("📊 Chart saved as finops_cost_breakdown.png")

```



Part 7: Complete FinOps Workflow

Run a full cycle: submit workloads → monitor → detect waste → autoscale → optimize cost.

```

1 # Cell 18: Full FinOps Optimization Workflow
2 print("📊 FULL FINOPS OPTIMIZATION WORKFLOW")
3 print("=" * 60)
4
5 # Step 1: Check initial state
6 print("\n📌 Initial cluster state:")

```

```

7 m = requests.get(f"{GATEWAY_URL}/cluster/metrics").json()
8 print(f"   GPUs: {m['total_gpus']} | Util: {m['avg_utilization']:.1f}% | Idle: {m['idle_gpus']}")
9
10 # Step 2: Submit heavy workloads to increase utilization
11 print("\n2 Submitting heavy workloads...")
12 for i in range(5):
13     requests.post(f"{GATEWAY_URL}/cluster/workloads/submit", json={
14         "workload_id": f"heavy-{i:03d}", "gpu_type_preferred": "T4", "gpu_count": 1, "duration_seconds": 600
15     })
16 m = requests.get(f"{GATEWAY_URL}/cluster/metrics").json()
17 print(f"   After load: Util: {m['avg_utilization']:.1f}% | Busy: {m['busy_gpus']}/{m['total_gpus']}")
18
19 # Step 3: Autoscaler evaluates
20 print("\n3 Autoscaler evaluation:")
21 decision = requests.post(f"{GATEWAY_URL}/autoscaler/evaluate").json()
22 print(f"   Decision: {decision['action']} - {decision['reason']}")
23
24 # Step 4: Cost snapshot
25 print("\n4 Cost analysis:")
26 snap = requests.post(f"{GATEWAY_URL}/cost/snapshot").json()
27 print(f"   Total cost/interval: ${snap['total_cost_usd']:.6f}")
28 print(f"   Waste: {snap['waste_pct']:.1f}%")
29
30 # Step 5: Get recommendations
31 print("\n5 Recommendations:")
32 recs = requests.post(f"{GATEWAY_URL}/cost/recommendations").json()
33 for rec in recs:
34     print(f"   [{rec['priority']}] {rec['type']}: savings ~{rec['estimated_savings_pct']}%")
35
36 # Step 6: Apply optimization - use spot for some workloads
37 print("\n6 Applying optimization: Switch to spot instances...")
38 for i in range(3):
39     requests.post(f"{GATEWAY_URL}/spot/request", json={
40         "instance_id": f"opt-spot-{i}", "gpu_type": "T4",
41         "gpu_count": 1, "max_price_per_hour": 0.15, "workload_id": f"heavy-{i:03d}"
42     })
43
44 savings = requests.get(f"{GATEWAY_URL}/spot/savings-report").json()
45 print(f"   Spot savings: ${savings['total_savings']:.4f} ({savings['savings_pct']:.1f}%)")
46
47 # Step 7: Complete workloads and record billing
48 print("\n7 Final billing:")
49 for i in range(5):
50     requests.post(f"{GATEWAY_URL}/cluster/workloads/heavy-{i:03d}/complete")
51     requests.post(f"{GATEWAY_URL}/billing/record", json={
52         "workload_id": f"heavy-{i:03d}", "gpu_type": "T4",
53         "gpu_count": 1, "duration_seconds": 600, "is_spot": i < 3
54     })
55
56 final = requests.get(f"{GATEWAY_URL}/billing/summary").json()
57 print(f"   Total spend: ${final['total_cost_usd']:.4f}")
58 print(f"   Total saved: ${final['total_savings_usd']:.4f}")
59 print(f"   Budget: {final['budget_utilization_pct']:.1f}% used")
60 print(f"\n✅ Workflow complete!")

```

🔗 FULL FINOPS OPTIMIZATION WORKFLOW

=====

- 1 Initial cluster state:
GPUs: 18 | Util: 71.8% | Idle: 2
- 2 Submitting heavy workloads...
After load: Util: 79.9% | Busy: 18/18
- 3 Autoscaler evaluation:
Decision: scale_up - Utilization 79.9% > threshold 70.0%
- 4 Cost analysis:
Total cost/interval: \$0.049722
Waste: 3.9%
- 5 Recommendations:
[MEDIUM] USE_SPOT: savings ~65.0%
[LOW] SCHEDULING: savings ~20.0%
- 6 Applying optimization: Switch to spot instances...
Spot savings: \$0.0357 (70.0%)
- 7 Final billing:

Total spend: \$5.4560
 Total saved: \$5.6604
 Budget: 5.5% used

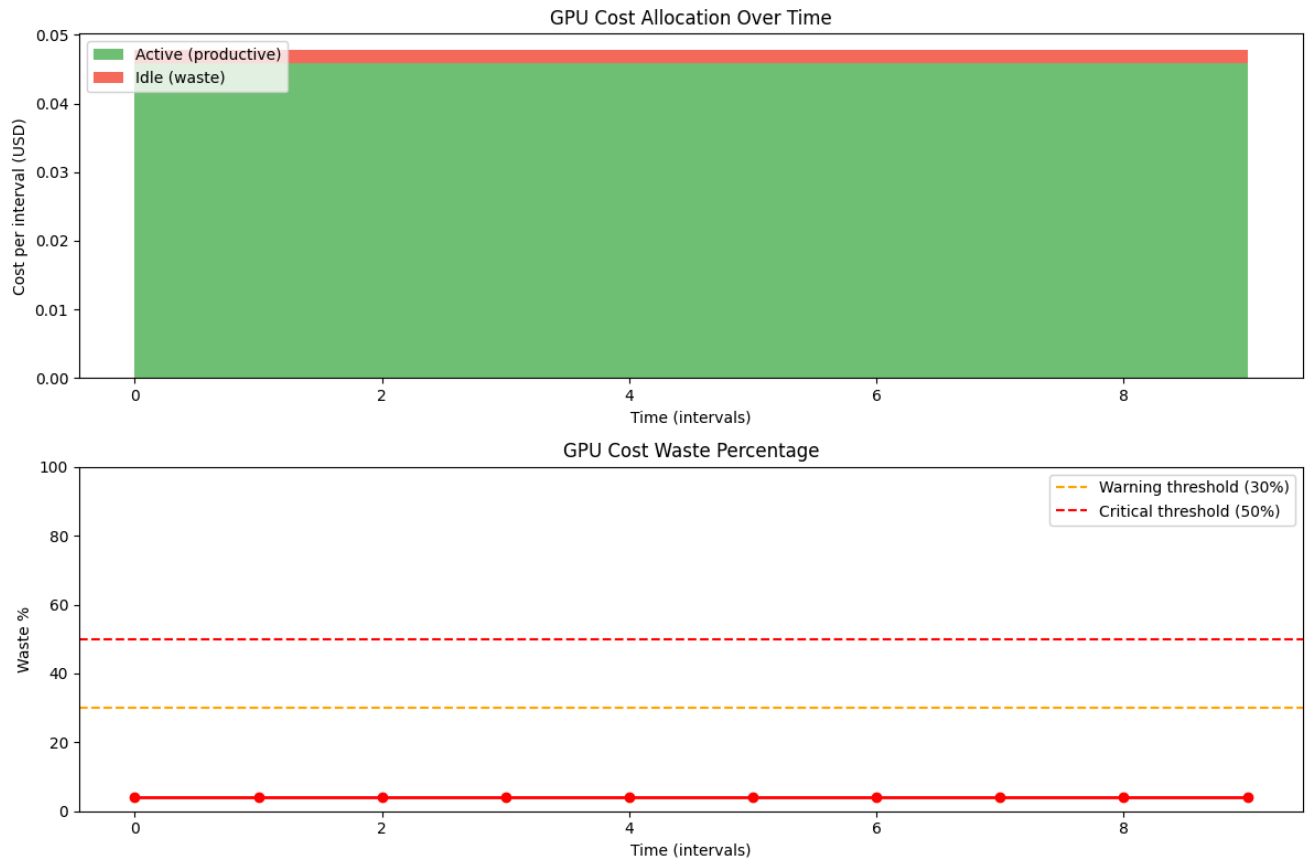
✓ Workflow complete!

```

1 # Cell 17: Time-series cost tracking
2 print("🔍 Collecting time-series data (10 snapshots)...")
3 ts_data = []
4 for i in range(10):
5     snap = requests.post(f"{GATEWAY_URL}/cost/snapshot").json()
6     ts_data.append({
7         'time': i,
8         'total_cost': snap['total_cost_usd'],
9         'idle_cost': snap['total_idle_cost_usd'],
10        'active_cost': snap['total_active_cost_usd'],
11        'waste_pct': snap['waste_pct'],
12    })
13    time.sleep(1)
14
15 df = pd.DataFrame(ts_data)
16
17 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
18
19 # Cost over time
20 ax1.stackplot(df['time'], df['active_cost'], df['idle_cost'],
21              labels=['Active (productive)', 'Idle (waste)'],
22              colors=['#4CAF50', '#F44336'], alpha=0.8)
23 ax1.set_xlabel('Time (intervals)')
24 ax1.set_ylabel('Cost per interval (USD)')
25 ax1.set_title('GPU Cost Allocation Over Time')
26 ax1.legend(loc='upper left')
27
28 # Waste percentage
29 ax2.plot(df['time'], df['waste_pct'], 'r-o', linewidth=2)
30 ax2.axhline(y=30, color='orange', linestyle='--', label='Warning threshold (30%)')
31 ax2.axhline(y=50, color='red', linestyle='--', label='Critical threshold (50%)')
32 ax2.set_xlabel('Time (intervals)')
33 ax2.set_ylabel('Waste %')
34 ax2.set_title('GPU Cost Waste Percentage')
35 ax2.legend()
36 ax2.set_ylim(0, 100)
37
38 plt.tight_layout()
39 plt.savefig('finops_timeseries.png', dpi=150, bbox_inches='tight')
40 plt.show()

```

Collecting time-series data (10 snapshots)...



Part 8: Real GPU Workload on Kaggle/Colab

Train actual models on this notebook's GPU, measure real utilization/time/power, then report cost back to the gateway.

This section:

1. Profiles the **real GPU** available (T4 on Kaggle, T4/V100 on Colab)
2. Trains **ResNet-18 on CIFAR-10** with live GPU monitoring
3. Compares **FP32 vs Mixed Precision (AMP)** to measure time & cost savings
4. Reports all costs back to the FinOps gateway

```
1 # Cell 19: Install dependencies & detect real GPU
2 !pip install -q torch torchvision pynvml
3
4 import torch
5 import torchvision
6 import torchvision.transforms as transforms
7 import torch.nn as nn
8 import torch.optim as optim
9 from torch.cuda.amp import autocast, GradScaler
10 import subprocess
11 import threading
12
13 try:
14     from pynvml import (nvmlInit, nvmlDeviceGetHandleByIndex,
15                        nvmlDeviceGetUtilizationRates, nvmlDeviceGetMemoryInfo,
16                        nvmlDeviceGetPowerUsage, nvmlDeviceGetTemperature)
17     PYNVML_AVAILABLE = True
18 except ImportError:
```

```

19 PYNVML_AVAILABLE = False
20 print("pynvml not available, will use torch.cuda fallback for monitoring")
21
22 # Detect GPU
23 if not torch.cuda.is_available():
24     raise RuntimeError("No GPU detected! Enable GPU in Kaggle/Colab settings.")
25
26 gpu_name = torch.cuda.get_device_name(0)
27 gpu_mem = torch.cuda.get_device_properties(0).total_memory / 1e9
28
29 # Map real GPU to pricing
30 GPU_PRICING = {"T4": 0.35, "A100": 3.67, "V100": 2.48, "P100": 1.46, "L4": 0.81}
31 detected_type = "T4" # default
32 for gtype in GPU_PRICING:
33     if gtype.lower() in gpu_name.lower():
34         detected_type = gtype
35         break
36
37 print(f"Real GPU Detected")
38 print(f"  Name:    {gpu_name}")
39 print(f"  Memory:  {gpu_mem:.1f} GB")
40 print(f"  Type:    {detected_type}")
41 print(f"  Pricing:  ${GPU_PRICING[detected_type]:.2f}/hr (on-demand)")
42 print(f"  CUDA:    {torch.version.cuda}")
43 print(f"  pynvml:  {'available' if PYNVML_AVAILABLE else 'NOT available'}")

```

```

Real GPU Detected
Name:    Tesla T4
Memory:  15.6 GB
Type:    T4
Pricing:  $0.35/hr (on-demand)
CUDA:    12.8
pynvml:  available

```

```

1 # Cell 2.5: Student Information Setup
2 # Nhập thông tin sinh viên của bạn ở đây
3 STUDENT_NAME = "Võ Thiên Phú" # <-- Điền tên của bạn
4 STUDENT_ID = "2A202600336"    # <-- Điền MSSV của bạn
5
6 # Tạo hàm hiển thị thông tin sinh viên
7 from IPython.display import display, HTML
8
9 def display_student_header():
10     """Hiển thị header với thông tin sinh viên"""
11     header = f"""
12     <div style="background: linear-gradient(90deg, #667eea 0%, #764ba2 100%);
13         color: white; padding: 15px; border-radius: 10px;
14         margin: 10px 0; font-family: Arial, sans-serif;">
15         <h2 style="margin: 0; color: white;"> GPU FinOps Lab - Student Information</h2>
16         <p style="margin: 5px 0 0 0; font-size: 16px;">
17             <strong>Họ và tên:</strong> {STUDENT_NAME} |
18             <strong>MSSV:</strong> {STUDENT_ID}
19         </p>
20     </div>
21     """
22     display(HTML(header))
23
24 # Hiển thị thông tin sinh viên
25 print("=" * 60)
26 print("⚠️  QUAN TRỌNG: Nhập thông tin sinh viên của bạn ở trên!")
27 print("  Thông tin này sẽ hiển thị trong tất cả screenshots")
28 print("=" * 60)
29 display_student_header()
30

```

```

=====
⚠️  QUAN TRỌNG: Nhập thông tin sinh viên của bạn ở trên!
  Thông tin này sẽ hiển thị trong tất cả screenshots
=====

```



GPU FinOps Lab - Student Information

Họ và tên: Võ Thiên Phú | MSSV: 2A202600336

```

1 # Cell 20: GPU Metrics Collection (works on Kaggle/Colab)
2 # Strategy: pynvml first (reliable Python lib), torch.cuda fallback
3
4 def get_gpu_metrics():
5     """Get GPU metrics inline. Called during training loop."""
6
7     # Method 1: pynvml (most reliable on Kaggle - pure Python, no subprocess)
8     if PYNVML_AVAILABLE:
9         try:
10             nvmlInit()
11             handle = nvmlDeviceGetHandleByIndex(0)
12             util = nvmlDeviceGetUtilizationRates(handle)
13             mem_info = nvmlDeviceGetMemoryInfo(handle)
14             try:
15                 power = nvmlDeviceGetPowerUsage(handle) / 1000.0 # mW -> W
16             except Exception:
17                 power = 0.0
18             try:
19                 temp = nvmlDeviceGetTemperature(handle, 0) # NVML_TEMPERATURE_GPU = 0
20             except Exception:
21                 temp = 0.0
22             return {
23                 'timestamp': time.time(),
24                 'gpu_util_pct': float(util.gpu),
25                 'mem_util_pct': float(util.memory),
26                 'mem_used_mb': mem_info.used / 1e6,
27                 'mem_total_mb': mem_info.total / 1e6,
28                 'power_watts': float(power),
29                 'temp_c': float(temp),
30             }
31         except Exception as e:
32             pass # Fall through to torch.cuda
33
34     # Method 2: torch.cuda (always works, limited metrics)
35     mem_used = torch.cuda.memory_allocated() / 1e6
36     mem_reserved = torch.cuda.memory_reserved() / 1e6
37     mem_total = torch.cuda.get_device_properties(0).total_memory / 1e6
38     util_approx = (mem_reserved / mem_total) * 100 if mem_total > 0 else 0
39     return {
40         'timestamp': time.time(),
41         'gpu_util_pct': util_approx,
42         'mem_util_pct': util_approx,
43         'mem_used_mb': mem_used,
44         'mem_total_mb': mem_total,
45         'power_watts': 0.0,
46         'temp_c': 0.0,
47     }
48
49 # === DIAGNOSTIC TEST ===
50 print("=" * 50)
51 print("GPU METRICS DIAGNOSTIC")
52 print("=" * 50)
53
54 # Test pynvml
55 print(f"\n1. pynvml available: {PYNVML_AVAILABLE}")
56 if PYNVML_AVAILABLE:
57     try:
58         nvmlInit()
59         handle = nvmlDeviceGetHandleByIndex(0)
60         util = nvmlDeviceGetUtilizationRates(handle)
61         mem = nvmlDeviceGetMemoryInfo(handle)
62         print(f"    pynvml works! GPU util={util.gpu}%, mem={mem.used/1e6:.0f}/{mem.total/1e6:.0f} MB")
63     try:
64         pwr = nvmlDeviceGetPowerUsage(handle) / 1000.0
65         print(f"    Power: {pwr:.1f}W")
66     except Exception as e:
67         print(f"    Power: not available ({e})")
68     try:
69         tmp = nvmlDeviceGetTemperature(handle, 0)
70         print(f"    Temp: {tmp}C")
71     except Exception as e:
72         print(f"    Temp: not available ({e})")
73 except Exception as e:
74     print(f"    pynvml FAILED: {e}")
75
76 # Test full function
77 print(f"\n2. get_gpu_metrics() test:")

```

```

78 test_result = get_gpu_metrics()
79 for k, v in sorted(test_result.items()):
80     if k != 'timestamp':
81         print(f"    {k}: {v}")
82
83 print(f"\n    Method: {'pynvml' if test_result.get('temp_c', 0) > 0 or (PYNVML_AVAILABLE and test_result.get('gpu_util_pct',
84 print(f"\nReady for training.")

```

```

=====
GPU METRICS DIAGNOSTIC
=====

```

```

1. pynvml available: True
   pynvml works! GPU util=0%, mem=472/16106 MB
   Power: 10.3W
   Temp: 42C

```

```

2. get_gpu_metrics() test:
   gpu_util_pct: 0.0
   mem_total_mb: 16106.12736
   mem_used_mb: 472.055808
   mem_util_pct: 0.0
   power_watts: 10.312
   temp_c: 42.0

```

```

   Method: pynvml

```

```

Ready for training.

```

```

1 # Cell 21: Prepare CIFAR-10 dataset and ResNet-18 model
2 transform_train = transforms.Compose([
3     transforms.RandomCrop(32, padding=4),
4     transforms.RandomHorizontalFlip(),
5     transforms.ToTensor(),
6     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
7 ])
8
9 trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform_train)
10 trainloader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=True, num_workers=2)
11
12 testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transforms.Compose([
13     transforms.ToTensor(),
14     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
15 ]))
16 testloader = torch.utils.data.DataLoader(testset, batch_size=256, shuffle=False, num_workers=2)
17
18 print(f"Dataset ready: {len(trainset)} train / {len(testset)} test images")
19 print(f"Batches per epoch: {len(trainloader)}")
20
21
22 def create_model():
23     """Create a fresh ResNet-18 for CIFAR-10."""
24     model = torchvision.models.resnet18(weights=None, num_classes=10)
25     model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
26     model.maxpool = nn.Identity()
27     return model.cuda()
28
29
30 def train_epoch_monitored(model, loader, optimizer, criterion, use_amp=False, sample_interval=10):
31     """Train one epoch with inline GPU metrics collection every N batches."""
32     model.train()
33     total_loss, correct, total = 0, 0, 0
34     scaler = GradScaler() if use_amp else None
35     metrics_samples = []
36     start = time.time()
37
38     for batch_idx, (inputs, targets) in enumerate(loader):
39         inputs, targets = inputs.cuda(), targets.cuda()
40         optimizer.zero_grad()
41
42         if use_amp:
43             with autocast():
44                 outputs = model(inputs)
45                 loss = criterion(outputs, targets)
46                 scaler.scale(loss).backward()
47                 scaler.step(optimizer)
48                 scaler.update()
49         else:

```

```

50         outputs = model(inputs)
51         loss = criterion(outputs, targets)
52         loss.backward()
53         optimizer.step()
54
55         total_loss += loss.item() * inputs.size(0)
56         _, predicted = outputs.max(1)
57         correct += predicted.eq(targets).sum().item()
58         total += inputs.size(0)
59
60         # Collect GPU metrics every N batches (inline, no threading)
61         if batch_idx % sample_interval == 0:
62             torch.cuda.synchronize()
63             sample = get_gpu_metrics()
64             metrics_samples.append(sample)
65
66         elapsed = time.time() - start
67         return total_loss / total, correct / total * 100, elapsed, metrics_samples
68
69
70 print("Model & training functions ready (inline monitoring every 10 batches)")

```

```

100%|██████████| 170M/170M [00:05<00:00, 32.2MB/s]
Dataset ready: 50000 train / 10000 test images
Batches per epoch: 391
Model & training functions ready (inline monitoring every 10 batches)

```

```

1 # Cell 22: Train FP32 (baseline) with inline GPU monitoring
2 NUM_EPOCHS = 3
3 price_per_hour = GPU_PRICING[detected_type]
4
5 print("=" * 60)
6 print("EXPERIMENT 1: FP32 Training (Baseline)")
7 print("=" * 60)
8
9 model_fp32 = create_model()
10 optimizer_fp32 = optim.SGD(model_fp32.parameters(), lr=0.1, momentum=0.9, weight_decay=5e-4)
11 criterion = nn.CrossEntropyLoss()
12
13 torch.cuda.synchronize()
14 torch.cuda.reset_peak_memory_stats()
15 fp32_start = time.time()
16
17 fp32_results = []
18 fp32_all_metrics = []
19 for epoch in range(NUM_EPOCHS):
20     loss, acc, t, epoch_metrics = train_epoch_monitored(model_fp32, trainloader, optimizer_fp32, criterion, use_amp=False)
21     fp32_results.append({"epoch": epoch+1, "loss": loss, "accuracy": acc, "time_s": t})
22     fp32_all_metrics.extend(epoch_metrics)
23     print(f"    Epoch {epoch+1}/{NUM_EPOCHS} | Loss: {loss:.4f} | Acc: {acc:.1f}% | Time: {t:.1f}s | Samples: {len(epoch_met
24
25 fp32_total_time = time.time() - fp32_start
26 fp32_peak_mem = torch.cuda.max_memory_allocated() / 1e9
27 fp32_cost = (fp32_total_time / 3600) * price_per_hour
28
29 print(f"\n    Total samples collected: {len(fp32_all_metrics)}")
30
31 # Build monitor dataframe
32 fp32_monitor_df = pd.DataFrame(fp32_all_metrics)
33 if not fp32_monitor_df.empty and 'timestamp' in fp32_monitor_df.columns:
34     fp32_monitor_df['elapsed'] = fp32_monitor_df['timestamp'] - fp32_monitor_df['timestamp'].iloc[0]
35
36 print(f"\n    FP32 Summary:")
37 print(f"    Total time:      {fp32_total_time:.1f}s")
38 print(f"    Peak memory:     {fp32_peak_mem:.2f} GB")
39 if not fp32_monitor_df.empty and 'gpu_util_pct' in fp32_monitor_df.columns:
40     print(f"    Avg GPU util:    {fp32_monitor_df['gpu_util_pct'].mean():.1f}%")
41     print(f"    Avg power:       {fp32_monitor_df['power_watts'].mean():.1f}W")
42     print(f"    Avg temperature: {fp32_monitor_df['temp_c'].mean():.1f}C")
43     print(f"    Max GPU util:    {fp32_monitor_df['gpu_util_pct'].max():.1f}%")
44 else:
45     print(f"    GPU util:        DataFrame empty! columns={list(fp32_monitor_df.columns) if not fp32_monitor_df.empty else '
46     if fp32_all_metrics:
47         print(f"    First sample keys: {list(fp32_all_metrics[0].keys())}")
48 print(f"    Estimated cost:  ${fp32_cost:.6f}")

```

```
=====
```

```
EXPERIMENT 1: FP32 Training (Baseline)
```

```
=====
```

```
Epoch 1/3 | Loss: 1.9126 | Acc: 32.9% | Time: 44.1s | Samples: 40
Epoch 2/3 | Loss: 1.3690 | Acc: 49.8% | Time: 41.0s | Samples: 40
Epoch 3/3 | Loss: 1.0686 | Acc: 61.8% | Time: 44.9s | Samples: 40
```

```
Total samples collected: 120
```

```
FP32 Summary:
```

```
Total time:      130.1s
Peak memory:     0.82 GB
Avg GPU util:    91.2%
Avg power:       66.2W
Avg temperature: 67.8C
Max GPU util:    98.0%
Estimated cost:  $0.012645
```

```
1 # Cell 23: Train Mixed Precision AMP (optimized) with inline GPU monitoring
2 print("=" * 60)
3 print("EXPERIMENT 2: Mixed Precision (AMP) Training")
4 print("=" * 60)
5
6 model_amp = create_model()
7 optimizer_amp = optim.SGD(model_amp.parameters(), lr=0.1, momentum=0.9, weight_decay=5e-4)
8
9 torch.cuda.synchronize()
10 torch.cuda.reset_peak_memory_stats()
11 amp_start = time.time()
12
13 amp_results = []
14 amp_all_metrics = []
15 for epoch in range(NUM_EPOCHS):
16     loss, acc, t, epoch_metrics = train_epoch_monitored(model_amp, trainloader, optimizer_amp, criterion, use_amp=True)
17     amp_results.append({"epoch": epoch+1, "loss": loss, "accuracy": acc, "time_s": t})
18     amp_all_metrics.extend(epoch_metrics)
19     print(f"    Epoch {epoch+1}/{NUM_EPOCHS} | Loss: {loss:.4f} | Acc: {acc:.1f}% | Time: {t:.1f}s | Samples: {len(epoch_met
20
21 amp_total_time = time.time() - amp_start
22 amp_peak_mem = torch.cuda.max_memory_allocated() / 1e9
23 amp_cost = (amp_total_time / 3600) * price_per_hour
24
25 print(f"\n    Total samples collected: {len(amp_all_metrics)}")
26
27 # Build monitor dataframe
28 amp_monitor_df = pd.DataFrame(amp_all_metrics)
29 if not amp_monitor_df.empty and 'timestamp' in amp_monitor_df.columns:
30     amp_monitor_df['elapsed'] = amp_monitor_df['timestamp'] - amp_monitor_df['timestamp'].iloc[0]
31
32 print(f"\n    AMP Summary:")
33 print(f"    Total time:      {amp_total_time:.1f}s")
34 print(f"    Peak memory:     {amp_peak_mem:.2f} GB")
35 if not amp_monitor_df.empty and 'gpu_util_pct' in amp_monitor_df.columns:
36     print(f"    Avg GPU util:    {amp_monitor_df['gpu_util_pct'].mean():.1f}%")
37     print(f"    Avg power:       {amp_monitor_df['power_watts'].mean():.1f}W")
38     print(f"    Avg temperature: {amp_monitor_df['temp_c'].mean():.1f}C")
39     print(f"    Max GPU util:    {amp_monitor_df['gpu_util_pct'].max():.1f}%")
40 else:
41     print(f"    GPU util:       DataFrame empty! columns={list(amp_monitor_df.columns) if not amp_monitor_df.empty else 'EM
42     if amp_all_metrics:
43         print(f"    First sample keys: {list(amp_all_metrics[0].keys())}")
44 print(f"    Estimated cost:  ${amp_cost:.6f}")
```

```
=====
```

```
EXPERIMENT 2: Mixed Precision (AMP) Training
```

```
=====
```

```
/tmp/ipykernel_2189/927799492.py:34: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.Gr
scaler = GradScaler() if use_amp else None
/tmp/ipykernel_2189/927799492.py:43: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.auto
with autocast():
```

```
Epoch 1/3 | Loss: 1.9897 | Acc: 29.8% | Time: 24.0s | Samples: 40
Epoch 2/3 | Loss: 1.4152 | Acc: 47.8% | Time: 23.4s | Samples: 40
Epoch 3/3 | Loss: 1.1470 | Acc: 58.9% | Time: 24.0s | Samples: 40
```

```
Total samples collected: 120
```

```
AMP Summary:
```

```
Total time:      71.4s
Peak memory:     0.60 GB
```

```

Avg GPU util:    69.6%
Avg power:       66.0W
Avg temperature: 76.6C
Max GPU util:    87.0%
Estimated cost:  $0.006940

```

```

1 # Cell 24: Compare FP32 vs AMP - FinOps Analysis
2 print("=" * 60)
3 print("FINOPS COMPARISON: FP32 vs Mixed Precision (AMP)")
4 print("=" * 60)
5
6 speedup = fp32_total_time / amp_total_time
7 cost_saving = fp32_cost - amp_cost
8 cost_saving_pct = (cost_saving / fp32_cost) * 100 if fp32_cost > 0 else 0
9 mem_saving = fp32_peak_mem - amp_peak_mem
10
11 print(f"\n{'Metric':<25} {'FP32':<15} {'AMP':<15} {'Improvement'}")
12 print("-" * 70)
13 print(f"{'Total Time':<25} {fp32_total_time:<15.1f} {amp_total_time:<15.1f} {speedup:.2f}x faster")
14 print(f"{'Peak Memory (GB)':<25} {fp32_peak_mem:<15.2f} {amp_peak_mem:<15.2f} {mem_saving:.2f} GB saved")
15 print(f"{'Cost (USD)':<25} ${fp32_cost:<14.6f} ${amp_cost:<14.6f} ${cost_saving:.6f} saved")
16 print(f"{'Cost Saving %':<25} {'---':<15} {'---':<15} {cost_saving_pct:.1f}%")
17
18 has_monitor_data = (not fp32_monitor_df.empty and 'gpu_util_pct' in fp32_monitor_df.columns
19                     and not amp_monitor_df.empty and 'gpu_util_pct' in amp_monitor_df.columns)
20
21 if has_monitor_data:
22     print(f"{'Avg GPU Util %':<25} {fp32_monitor_df['gpu_util_pct'].mean():<15.1f} {amp_monitor_df['gpu_util_pct'].mean():<15.1f}")
23     print(f"{'Avg Power (W)':<25} {fp32_monitor_df['power_watts'].mean():<15.1f} {amp_monitor_df['power_watts'].mean():<15.1f}")
24
25 # Extrapolate to larger scale
26 print(f"\n--- Extrapolated Savings at Scale ---")
27 for scale_name, hours in [("1 day training", 24), ("1 week training", 168), ("1 month training", 720)]:
28     fp32_scaled = hours * price_per_hour
29     amp_scaled = (hours / speedup) * price_per_hour
30     saved = fp32_scaled - amp_scaled
31     print(f"    {scale_name}: FP32=${fp32_scaled:.2f} vs AMP=${amp_scaled:.2f} -> SAVE ${saved:.2f}")
32
33 # Visualization
34 fig, axes = plt.subplots(1, 3, figsize=(16, 5))
35
36 # Bar: Time comparison
37 axes[0].bar(['FP32', 'AMP'], [fp32_total_time, amp_total_time], color=['#F44336', '#4CAF50'])
38 axes[0].set_ylabel('Time (seconds)')
39 axes[0].set_title(f'Training Time ({speedup:.2f}x speedup)')
40
41 # Bar: Cost comparison
42 axes[1].bar(['FP32', 'AMP'], [fp32_cost, amp_cost], color=['#F44336', '#4CAF50'])
43 axes[1].set_ylabel('Cost (USD)')
44 axes[1].set_title(f'Training Cost (save {cost_saving_pct:.1f}%)')
45
46 # Line: GPU utilization over time (both experiments) or epoch time
47 if has_monitor_data:
48     if not fp32_monitor_df.empty:
49         axes[2].plot(fp32_monitor_df['elapsed'], fp32_monitor_df['gpu_util_pct'], 'r-', alpha=0.7, label='FP32')
50     if not amp_monitor_df.empty:
51         axes[2].plot(amp_monitor_df['elapsed'], amp_monitor_df['gpu_util_pct'], 'g-', alpha=0.7, label='AMP')
52     axes[2].set_xlabel('Time (s)')
53     axes[2].set_ylabel('GPU Utilization %')
54     axes[2].set_title('Real GPU Utilization')
55     axes[2].legend()
56     axes[2].set_ylim(0, 100)
57 else:
58     # Fallback: show epoch times
59     epochs = list(range(1, NUM_EPOCHS + 1))
60     axes[2].plot(epochs, [r['time_s'] for r in fp32_results], 'r-o', label='FP32')
61     axes[2].plot(epochs, [r['time_s'] for r in amp_results], 'g-o', label='AMP')
62     axes[2].set_xlabel('Epoch')
63     axes[2].set_ylabel('Time (s)')
64     axes[2].set_title('Time per Epoch')
65     axes[2].legend()
66
67 plt.tight_layout()
68 plt.savefig('real_gpu_comparison.png', dpi=150, bbox_inches='tight')
69 plt.show()

```


=====

FINOPS COMPARISON: FP32 vs Mixed Precision (AMP)

=====

Metric	FP32	AMP	Improvement
Total Time	130.1	71.4	1.82x faster
Peak Memory (GB)	0.82	0.60	0.22 GB saved
Cost (USD)	\$0.012645	\$0.006940	\$0.005706 saved
Cost Saving %	---	---	45.1%
Avg GPU Util %	91.2	69.6	
Avg Power (W)	66.2	66.0	

--- Extrapolated Savings at Scale ---

1 day training: FP32=\$8.40 vs AMP=\$4.61 -> SAVE \$3.79

1 week training: FP32=\$58.80 vs AMP=\$32.27 -> SAVE \$26.53

1 month training: FP32=\$252.00 vs AMP=\$138.30 -> SAVE \$113.70



```

1 # Cell 25: Report real GPU costs back to the FinOps Gateway
2 print("=" * 60)
3 print("REPORTING REAL GPU COSTS TO FINOPS GATEWAY")
4 print("=" * 60)
5
6 # Report FP32 workload
7 fp32_report = requests.post(f"{GATEWAY_URL}/billing/record", json={
8     "workload_id": "real-gpu-resnet18-fp32",
9     "gpu_type": detected_type,
10    "gpu_count": 1,
11    "duration_seconds": fp32_total_time,
12    "is_spot": False,
13    "project": "real-gpu-lab",
14 }).json()
15 print(f"\n  FP32 workload reported:")
16 print(f"    Cost: ${fp32_report['total_cost_usd']:.6f} | Rate: ${fp32_report['rate_per_hour']:.4f}/hr")
17
18 # Report AMP workload (as spot - demonstrating cost optimization)
19 amp_report = requests.post(f"{GATEWAY_URL}/billing/record", json={
20     "workload_id": "real-gpu-resnet18-amp",
21     "gpu_type": detected_type,
22     "gpu_count": 1,
23     "duration_seconds": amp_total_time,
24     "is_spot": True,
25     "project": "real-gpu-lab",
26 }).json()
27 print(f"\n  AMP workload reported (as spot):")
28 print(f"    Cost: ${amp_report['total_cost_usd']:.6f} | Saved: ${amp_report['savings_usd']:.6f}")
29
30 # Submit workload info to cluster tracker
31 for wl_name, wl_time in [("real-gpu-resnet18-fp32", fp32_total_time), ("real-gpu-resnet18-amp", amp_total_time)]:
32     requests.post(f"{GATEWAY_URL}/cluster/workloads/submit", json={
33         "workload_id": wl_name,
34         "gpu_type_preferred": detected_type,

```

```

35     "gpu_count": 1,
36     "duration_seconds": int(wl_time),
37 })
38
39 # Get updated billing summary including real GPU data
40 print(f"\n--- Updated FinOps Billing (incl. real GPU) ---")
41 summary = requests.get(f"{GATEWAY_URL}/billing/summary", params={"project": "real-gpu-lab"}).json()
42 print(f"    Project:        real-gpu-lab")
43 print(f"    Total Cost:      ${summary['total_cost_usd']:.6f}")
44 print(f"    Total Savings:    ${summary['total_savings_usd']:.6f}")
45 print(f"    Workloads:        {summary['total_workloads']}")
46
47 # Push real GPU metrics as a cost snapshot
48 snap = requests.post(f"{GATEWAY_URL}/cost/snapshot").json()
49 print(f"\n    Cost snapshot taken: waste={snap['waste_pct']:.1f}%")
50
51 # Get final dashboard with everything
52 print(f"\n--- FINAL DASHBOARD (Mock + Real GPU) ---")
53 dash = requests.get(f"{GATEWAY_URL}/cost/dashboard").json()
54 bs = dash.get('billing_summary', {})
55 print(f"    Total Platform Cost:  ${bs.get('total_cost_usd', 0):.4f}")
56 print(f"    Total Savings:        ${bs.get('total_savings_usd', 0):.4f}")
57 print(f"    Budget Utilization:    {bs.get('budget_utilization_pct', 0):.1f}%")
58 print(f"    Alert:                 {bs.get('alert', 'N/A')}")

```

```

=====
REPORTING REAL GPU COSTS TO FINOPS GATEWAY
=====

```

```

FP32 workload reported:
Cost: $0.012600 | Rate: $0.3500/hr

AMP workload reported (as spot):
Cost: $0.002100 | Saved: $0.004900

```

```

--- Updated FinOps Billing (incl. real GPU) ---

```

```

Project:        real-gpu-lab
Total Cost:      $0.014700
Total Savings:    $0.004900
Workloads:        2

```

```

Cost snapshot taken: waste=17.8%

```

```

--- FINAL DASHBOARD (Mock + Real GPU) ---

```

```

Total Platform Cost:  $5.4560
Total Savings:        $5.6604
Budget Utilization:    5.5%
Alert:                 OK

```

```

1 # Cell 26: Real GPU Monitoring Visualization
2 print("Real GPU Telemetry During Training")
3 print("=" * 60)
4
5 has_monitor_data = (not fp32_monitor_df.empty and 'gpu_util_pct' in fp32_monitor_df.columns
6                     and not amp_monitor_df.empty and 'gpu_util_pct' in amp_monitor_df.columns)
7
8 if has_monitor_data:
9     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
10
11     # GPU Utilization
12     for label, df, color in [("FP32", fp32_monitor_df, '#F44336'), ("AMP", amp_monitor_df, '#4CAF50')]:
13         if not df.empty:
14             axes[0, 0].plot(df['elapsed'], df['gpu_util_pct'], color=color, alpha=0.8, label=label)
15     axes[0, 0].set_title('GPU Utilization %')
16     axes[0, 0].set_ylabel('%')
17     axes[0, 0].legend()
18     axes[0, 0].set_ylim(0, 100)
19
20     # Memory Usage
21     for label, df, color in [("FP32", fp32_monitor_df, '#F44336'), ("AMP", amp_monitor_df, '#4CAF50')]:
22         if not df.empty:
23             axes[0, 1].plot(df['elapsed'], df['mem_used_mb'] / 1024, color=color, alpha=0.8, label=label)
24     axes[0, 1].set_title('GPU Memory Used (GB)')
25     axes[0, 1].set_ylabel('GB')
26     axes[0, 1].legend()
27
28     # Power Draw
29     for label, df, color in [("FP32", fp32_monitor_df, '#F44336'), ("AMP", amp_monitor_df, '#4CAF50')]:

```

```

30         if not df.empty:
31             axes[1, 0].plot(df['elapsed'], df['power_watts'], color=color, alpha=0.8, label=label)
32         axes[1, 0].set_title('Power Draw (Watts)')
33         axes[1, 0].set_ylabel('Watts')
34         axes[1, 0].set_xlabel('Time (s)')
35         axes[1, 0].legend()
36
37     # Temperature
38     for label, df, color in [("FP32", fp32_monitor_df, '#F44336'), ("AMP", amp_monitor_df, '#4CAF50')]:
39         if not df.empty:
40             axes[1, 1].plot(df['elapsed'], df['temp_c'], color=color, alpha=0.8, label=label)
41         axes[1, 1].set_title('GPU Temperature (C)')
42         axes[1, 1].set_ylabel('C')
43         axes[1, 1].set_xlabel('Time (s)')
44         axes[1, 1].legend()
45
46     plt.suptitle(f'Real GPU Telemetry: {gpu_name}', fontsize=14, fontweight='bold')
47     plt.tight_layout()
48     plt.savefig('real_gpu_telemetry.png', dpi=150, bbox_inches='tight')
49     plt.show()
50 else:
51     print("    GPU telemetry not available (monitor collected 0 samples).")
52     print("    Showing epoch-level metrics instead.\n")
53
54 # Cost per epoch chart (always works)
55 fig, ax = plt.subplots(figsize=(10, 5))
56 fp32_epoch_costs = [(r['time_s'] / 3600) * price_per_hour for r in fp32_results]
57 amp_epoch_costs = [(r['time_s'] / 3600) * price_per_hour for r in amp_results]
58 epochs = list(range(1, NUM_EPOCHS + 1))
59
60 x = range(len(epochs))
61 width = 0.35
62 bars1 = ax.bar([i - width/2 for i in x], fp32_epoch_costs, width, label='FP32', color='#F44336')
63 bars2 = ax.bar([i + width/2 for i in x], amp_epoch_costs, width, label='AMP', color='#4CAF50')
64 ax.set_xlabel('Epoch')
65 ax.set_ylabel('Cost (USD)')
66 ax.set_title('Cost per Epoch: FP32 vs AMP')
67 ax.set_xticks(x)
68 ax.set_xticklabels(epochs)
69 ax.legend()
70
71 for bar in bars1:
72     ax.text(bar.get_x() + bar.get_width()/2., bar.get_height(), f'${bar.get_height():.5f}',
73             ha='center', va='bottom', fontsize=8)
74 for bar in bars2:
75     ax.text(bar.get_x() + bar.get_width()/2., bar.get_height(), f'${bar.get_height():.5f}',
76             ha='center', va='bottom', fontsize=8)
77
78 plt.tight_layout()
79 plt.savefig('cost_per_epoch.png', dpi=150, bbox_inches='tight')
80 plt.show()
81
82 print("\nCharts saved: cost_per_epoch.png" + ("", real_gpu_telemetry.png" if has_monitor_data else ""))

```


Real GPU Telemetry During Training

=====

Real GPU Telemetry: Tesla T4

GPU Utilization %

GPU Memory Used (GB)

100

1.644

1.642

1.640

1.638

1.636

1.634

1.632

FP32
AMP

Part 8.5: Advanced GPU Cost Optimization

Apply advanced FinOps techniques: multi-GPU cost analysis, project forecasting, and optimization strategy prioritization.

This section extends Part 8 with:

1. **Multi-GPU Cost Analysis** - Compare scaling efficiency and determine optimal GPU count
2. **Project Cost Forecasting** - Forecast multi-phase project costs with confidence intervals
3. **Optimization Opportunity Analysis** - Prioritize and combine optimization strategies

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy import stats
5
6 print("=" * 60)
7 print("EXERCISE 8.5.1: Multi-GPU Cost Analysis")
8 print("=" * 60)
9
10 GPU_PRICING = {"T4": 0.35, "A100": 3.67, "V100": 2.48, "P100": 1.46, "L4": 0.81}
11
12 def analyze_multi_gpu_cost(base_time_hours, gpu_type, gpu_counts, scaling_factors=None):
13     """
14     Analyze costs for different GPU counts with realistic scaling.
15     Uses Amdahl's law-inspired sub-linear scaling for efficiency.
16     """
17     price = GPU_PRICING.get(gpu_type, 3.67)
18
19     if scaling_factors is None:
20         # Realistic scaling: 2 GPUs = 1.8x, 4 = 3.4x, 8 = 6.5x (not linear)
21         scaling_factors = {1: 1.0, 2: 1.8, 4: 3.4, 8: 6.5}
22
23     results = []
24     for n in gpu_counts:
25         sf = scaling_factors.get(n, n ** 0.85)
26         scaled_time = base_time_hours / sf
27         total_cost = scaled_time * price * n
28         cost_per_unit = total_cost / sf # cost per "effective" GPU-hour
29         efficiency = sf / n * 100 # parallel efficiency %
30
31         results.append({
32             'gpu_count': n,
33             'time_hours': scaled_time,
34             'total_cost': total_cost,
35             'cost_per_gpu_hour': price * n,
36             'scaling_factor': sf,
37             'parallel_efficiency': efficiency,
38             'cost_per_perf_unit': cost_per_unit,
39         })
40
41     df = pd.DataFrame(results)
42
43     # Find optimal (lowest total cost)
44     idx_optimal = df['total_cost'].idxmin()
45
46     return {'table': df, 'optimal_gpu_count': df.loc[idx_optimal, 'gpu_count'],
47           'optimal_time': df.loc[idx_optimal, 'time_hours'],
48           'optimal_cost': df.loc[idx_optimal, 'total_cost']}
49
50 # Test
51 base_training_time = 2.0
52 test_gpu_counts = [1, 2, 4, 8]
53 result = analyze_multi_gpu_cost(base_training_time, "A100", test_gpu_counts)
54 df = result['table']
55
56 print(f"\n{'GPU#':<8}{ 'Time(h)':<10}{ 'Total Cost':<14}{ '$/GPU/hr':<12}{ 'Speedup':<10}{ 'Efficiency%':<12}{ 'Optimal?':<10}")
57 print("-" * 78)
58 for _, row in df.iterrows():
59     optimal = " <-- OPTIMAL" if row['gpu_count'] == result['optimal_gpu_count'] else ""

```

```

60 print(f"{int(row['gpu_count']):<8}{row['time_hours']:<10.2f}${row['total_cost']:<13.2f}${row['cost_per_gpu_hour']:<11.2f}
61      f"{row['scaling_factor']:<10.1f}x{row['parallel_efficiency']:<11.1f}{optimal}")
62
63 print(f"\n>>> Optimal configuration: {int(result['optimal_gpu_count'])} GPUs")
64 print(f"      Time: {result['optimal_time']:.2f}h | Cost: ${result['optimal_cost']:.2f}")

```

EXERCISE 8.5.1: Multi-GPU Cost Analysis

GPU#	Time(h)	Total Cost	\$/GPU/hr	Speedup	Efficiency%	Optimal?
1	2.00	\$7.34	\$3.67	1.0	x100.0	<-- OPTIMAL
2	1.11	\$8.16	\$7.34	1.8	x90.0	
4	0.59	\$8.64	\$14.68	3.4	x85.0	
8	0.31	\$9.03	\$29.36	6.5	x81.2	

```

>>> Optimal configuration: 1 GPUs
      Time: 2.00h | Cost: $7.34

```

```

1 print("=" * 60)
2 print("EXERCISE 8.5.2: Project Cost Forecasting")
3 print("=" * 60)
4
5 def forecast_project_cost(phases, contingency_pct=20, confidence_level=0.95):
6     """
7     Forecast multi-phase project costs with confidence intervals.
8     Uses log-normal distribution for cost uncertainty.
9     """
10    phase_details = []
11    total_base = 0
12
13    print(f"\n{'Phase':<25}{'GPU':<8}{'#GPU':<6}{'Hours':<8}{'Base Cost':<12}{'Uncertainty'}")
14    print("-" * 75)
15
16    for p in phases:
17        price = GPU_PRICING.get(p['gpu_type'], 3.67)
18        base = price * p['gpu_count'] * p['duration_hours']
19        total_base += base
20
21        # Log-normal uncertainty model
22        sigma = p['uncertainty_pct']
23        z = stats.norm.ppf(confidence_level)
24        lower = base * np.exp(-z * sigma)
25        upper = base * np.exp(z * sigma)
26
27        phase_details.append({
28            'name': p['name'],
29            'gpu_type': p['gpu_type'],
30            'gpu_count': p['gpu_count'],
31            'duration_hours': p['duration_hours'],
32            'base_cost': base,
33            'lower': lower,
34            'upper': upper,
35            'uncertainty_pct': p['uncertainty_pct'],
36        })
37        print(f"{p['name']:<25}{p['gpu_type']:<8}{p['gpu_count']:<6}{p['duration_hours']:<8}"
38              f"${base:<11.2f} +/-{p['uncertainty_pct']*100:.0f}%")
39
40    # Confidence interval for total
41    combined_sigma = np.sqrt(sum(p['uncertainty_pct']**2 for p in phases))
42    z = stats.norm.ppf(confidence_level)
43    contingency = total_base * (contingency_pct / 100)
44
45    total_lower = total_base * np.exp(-z * combined_sigma)
46    total_upper = total_base * np.exp(z * combined_sigma)
47    expected = total_base + contingency
48
49    print(f"\n{'='*50}")
50    print(f"      Base cost:          ${total_base:.2f}")
51    print(f"      Contingency ({contingency_pct}%):    +${contingency:.2f}")
52    print(f"      Expected total:      ${expected:.2f}")
53    print(f"      {confidence_level*100:.0f}% CI:        ${total_lower+contingency:.2f} - ${total_upper+contingency:.2f}")
54    print(f"      Best case:          ${total_lower:.2f}")
55    print(f"      Worst case:         ${total_upper+contingency*2:.2f}")
56
57    return {
58        'base_cost': total_base,

```

```

59     'contingency': contingency,
60     'expected_total': expected,
61     'ci_lower': total_lower + contingency,
62     'ci_upper': total_upper + contingency,
63     'phases': phase_details,
64 }
65
66 # Example project
67 example_project = [
68     {"name": "Data Preparation", "gpu_type": "T4", "gpu_count": 1, "duration_hours": 40, "uncertainty_pct": 0.15},
69     {"name": "Model Training", "gpu_type": "A100", "gpu_count": 4, "duration_hours": 120, "uncertainty_pct": 0.25},
70     {"name": "Hyperparameter Tuning", "gpu_type": "A100", "gpu_count": 8, "duration_hours": 60, "uncertainty_pct": 0.30},
71     {"name": "Model Evaluation", "gpu_type": "T4", "gpu_count": 2, "duration_hours": 20, "uncertainty_pct": 0.10},
72 ]
73
74 forecast = forecast_project_cost(example_project)

```

```

=====
EXERCISE 8.5.2: Project Cost Forecasting
=====

```

Phase	GPU	#GPU	Hours	Base Cost	Uncertainty
Data Preparation	T4	1	40	\$14.00	+/-15%
Model Training	A100	4	120	\$1761.60	+/-25%
Hyperparameter Tuning	A100	8	60	\$1761.60	+/-30%
Model Evaluation	T4	2	20	\$14.00	+/-10%

```

=====
Base cost:          $3551.20
Contingency (20%):  +$710.24
Expected total:     $4261.44
95% CI:             $2460.58 - $7915.16
Best case:          $1750.34
Worst case:         $8625.40

```

```

1 print("=" * 60)
2 print("EXERCISE 8.5.3: Advanced Optimization Opportunity Analysis")
3 print("=" * 60)
4
5 def analyze_optimization_opportunities(current_config, optimization_strategies):
6     """
7     Prioritize optimization strategies by savings/effort ratio,
8     considering risk and dependencies.
9     """
10    effort_map = {"LOW": 1, "MEDIUM": 2, "HIGH": 3}
11
12    # Calculate priority score: (savings * (4 - risk_score)) / effort
13    # Higher score = higher priority
14    for s in optimization_strategies:
15        risk_score = effort_map.get(s['risk_level'], 2)
16        effort_score = effort_map.get(s['implementation_effort'], 2)
17        s['priority_score'] = (s['savings_pct'] * (5 - risk_score)) / effort_score
18
19    # Sort by priority score descending
20    sorted_strategies = sorted(optimization_strategies, key=lambda x: x['priority_score'], reverse=True)
21
22    # Calculate cumulative savings
23    cumulative = 1.0
24    print(f"\n{'Rank':<6}{ 'Strategy':<35}{ 'Savings':<10}{ 'Effort':<10}{ 'Risk':<8}{ 'Score':<8}{ 'Cumul.Savings'}")
25    print("-" * 95)
26    for i, s in enumerate(sorted_strategies, 1):
27        cumulative *= (1 - s['savings_pct'])
28        total_savings = (1 - cumulative) * 100
29        print(f"{i:<6}{s['name']:<35}{s['savings_pct']*100:<9.0f}%{s['implementation_effort']:<10}{s['risk_level']:<8}"
30              f"{s['priority_score']:<8.2f}{total_savings:<.1f}%")
31
32    total_savings_pct = (1 - cumulative) * 100
33
34    # Quick wins (LOW effort, LOW risk)
35    quick_wins = [s['name'] for s in sorted_strategies
36                  if s['implementation_effort'] == 'LOW' and s['risk_level'] == 'LOW']
37
38    # High impact (HIGH savings)
39    high_impact = [s['name'] for s in sorted_strategies if s['savings_pct'] >= 0.20]
40
41    print(f"\n>>> Total potential savings: {total_savings_pct:<.1f}%")
42    print(f">>> Quick wins (implement first): {', '.join(quick_wins)}")

```

```

43     print(f">>> High impact strategies: {'', '.join(high_impact)}")
44
45     return {'ranked': sorted_strategies, 'total_savings_pct': total_savings_pct,
46           'quick_wins': quick_wins, 'high_impact': high_impact}
47
48 # Example
49 example_strategies = [
50     {"name": "Switch to Mixed Precision (AMP)", "savings_pct": 0.25, "implementation_effort": "LOW", "risk_level": "LOW"},
51     {"name": "Use Spot Instances", "savings_pct": 0.60, "implementation_effort": "MEDIUM", "risk_level": "HIGH"},
52     {"name": "Optimize Batch Size", "savings_pct": 0.15, "implementation_effort": "LOW", "risk_level": "LOW"},
53     {"name": "Implement Early Stopping", "savings_pct": 0.20, "implementation_effort": "MEDIUM", "risk_level": "LOW"},
54     {"name": "Switch to More Efficient GPU", "savings_pct": 0.40, "implementation_effort": "HIGH", "risk_level": "MEDIUM"}
55 ]
56
57 current_config = {
58     "gpu_type": "A100", "gpu_count": 4, "duration_hours": 100,
59     "precision": "FP32", "instance_type": "on-demand",
60 }
61
62 opt_result = analyze_optimization_opportunities(current_config, example_strategies.copy())

```

```

=====
EXERCISE 8.5.3: Advanced Optimization Opportunity Analysis
=====

```

Rank	Strategy	Savings	Effort	Risk	Score	Cumul.Savings
1	Switch to Mixed Precision (AMP)	25	%LOW	LOW	1.00	25.0%
2	Use Spot Instances	60	%MEDIUM	HIGH	0.60	70.0%
3	Optimize Batch Size	15	%LOW	LOW	0.60	74.5%
4	Switch to More Efficient GPU	40	%HIGH	MEDIUM	0.40	84.7%
5	Implement Early Stopping	20	%MEDIUM	LOW	0.40	87.8%

```

>>> Total potential savings: 87.8%
>>> Quick wins (implement first): Switch to Mixed Precision (AMP), Optimize Batch Size
>>> High impact strategies: Switch to Mixed Precision (AMP), Use Spot Instances, Switch to More Efficient GPU, Implement Early S

```

```

1 print("=" * 60)
2 print("EXERCISE 8.5.4: Integrated Cost Dashboard")
3 print("=" * 60)
4
5 def create_advanced_finops_dashboard(multi_gpu_analysis, project_forecast, optimization_recommendations):
6     """
7     Create 6-panel integrated dashboard.
8     """
9     fig, axes = plt.subplots(2, 3, figsize=(18, 10))
10
11     # 1. Multi-GPU Cost Curve
12     df = multi_gpu_analysis['table']
13     ax = axes[0, 0]
14     bars = ax.bar(df['gpu_count'].astype(str), df['total_cost'], color='#2196F3', alpha=0.8)
15     ax.bar(df[df['gpu_count'] == multi_gpu_analysis['optimal_gpu_count']]['gpu_count'].astype(str),
16           df[df['gpu_count'] == multi_gpu_analysis['optimal_gpu_count']]['total_cost'],
17           color='#4CAF50', alpha=0.9, label='Optimal')
18     ax.set_xlabel('GPU Count')
19     ax.set_ylabel('Total Cost (USD)')
20     ax.set_title('Multi-GPU Cost Analysis')
21     ax.legend()
22
23     # 2. Scaling Efficiency
24     ax = axes[0, 1]
25     ax.plot(df['gpu_count'], df['scaling_factor'], 'o-', color='#FF9800', linewidth=2, markersize=8)
26     ax.plot(df['gpu_count'], df['gpu_count'], '--', color='gray', alpha=0.5, label='Ideal linear')
27     ax.fill_between(df['gpu_count'], df['scaling_factor'], df['gpu_count'], alpha=0.3, color='#FF9800')
28     ax.set_xlabel('GPU Count')
29     ax.set_ylabel('Speedup Factor')
30     ax.set_title('Scaling Efficiency')
31     ax.legend()
32
33     # 3. Project Forecast CI
34     ax = axes[0, 2]
35     phases = project_forecast['phases']
36     names = [p['name'][:12] for p in phases]
37     costs = [p['base_cost'] for p in phases]
38     lowers = [p['base_cost'] - p['lower'] for p in phases]
39     uppers = [p['upper'] - p['base_cost'] for p in phases]
40     x = range(len(phases))

```

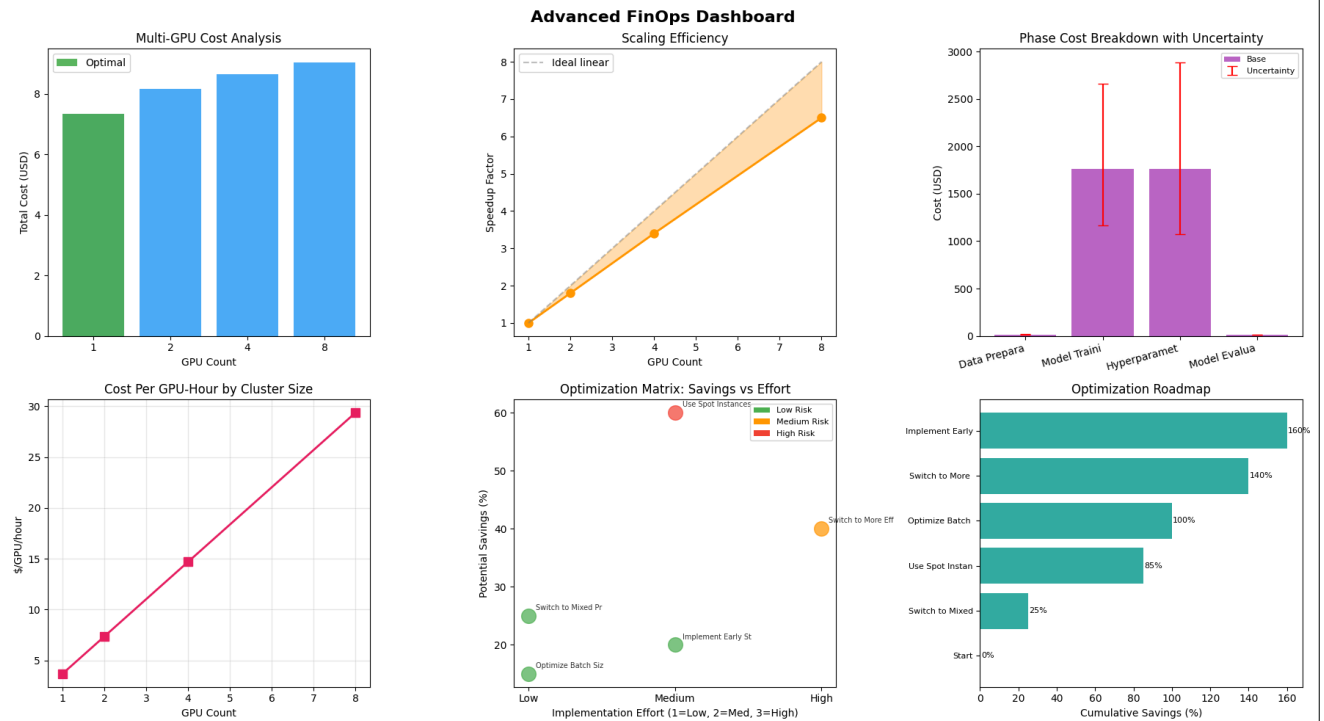


```

41 ax.bar(x, costs, color='#9C27B0', alpha=0.7, label='Base')
42 ax.errorbar(x, costs, yerr=[lowers, uppers], fmt='none', color='red', capsize=5, label='Uncertainty')
43 ax.set_xticks(x)
44 ax.set_xticklabels(names, rotation=15, ha='right')
45 ax.set_ylabel('Cost (USD)')
46 ax.set_title('Phase Cost Breakdown with Uncertainty')
47 ax.legend(fontsize=8)
48
49 # 4. Cost per GPU-hour across GPU counts
50 ax = axes[1, 0]
51 ax.plot(df['gpu_count'], df['cost_per_gpu_hour'], 's-', color='#E91E63', linewidth=2, markersize=8)
52 ax.set_xlabel('GPU Count')
53 ax.set_ylabel('$/GPU/hour')
54 ax.set_title('Cost Per GPU-Hour by Cluster Size')
55 ax.grid(True, alpha=0.3)
56
57 # 5. Optimization: Savings vs Effort scatter
58 ax = axes[1, 1]
59 ranked = optimization_recommendations['ranked']
60 effort_map = {"LOW": 1, "MEDIUM": 2, "HIGH": 3}
61 risk_map = {"LOW": 1, "MEDIUM": 2, "HIGH": 3}
62 colors = {'LOW': '#4CAF50', 'MEDIUM': '#FF9800', 'HIGH': '#F44336'}
63
64 for s in ranked:
65     ax.scatter(effort_map[s['implementation_effort']], s['savings_pct']*100,
66               c=colors[s['risk_level']], s=200, alpha=0.7)
67     ax.annotate(s['name'][:18], (effort_map[s['implementation_effort']] + 0.05, s['savings_pct']*100 + 1),
68               fontsize=7, alpha=0.8)
69 ax.set_xlabel('Implementation Effort (1=Low, 2=Med, 3=High)')
70 ax.set_ylabel('Potential Savings (%)')
71 ax.set_title('Optimization Matrix: Savings vs Effort')
72 from matplotlib.patches import Patch
73 legend_elements = [Patch(facecolor=colors['LOW'], label='Low Risk'),
74                    Patch(facecolor=colors['MEDIUM'], label='Medium Risk'),
75                    Patch(facecolor=colors['HIGH'], label='High Risk')]
76 ax.legend(handles=legend_elements, loc='upper right', fontsize=8)
77 ax.set_xticks([1, 2, 3])
78 ax.set_xticklabels(['Low', 'Medium', 'High'])
79
80 # 6. Cumulative Savings Roadmap
81 ax = axes[1, 2]
82 ranked = optimization_recommendations['ranked']
83 cumulative = [0]
84 labels = ['Start']
85 current = 0
86 for s in ranked:
87     current += s['savings_pct'] * 100
88     cumulative.append(current)
89     labels.append(s['name'][:15])
90
91 ax.barh(range(len(cumulative)), cumulative, color='#009688', alpha=0.8)
92 ax.set_yticks(range(len(labels)))
93 ax.set_yticklabels(labels, fontsize=8)
94 ax.set_xlabel('Cumulative Savings (%)')
95 ax.set_title('Optimization Roadmap')
96 for i, v in enumerate(cumulative):
97     ax.text(v + 0.5, i, f'{v:.0f}%', va='center', fontsize=8)
98
99 plt.suptitle('Advanced FinOps Dashboard', fontsize=16, fontweight='bold')
100 plt.tight_layout()
101 plt.savefig('advanced_finops_dashboard.png', dpi=150, bbox_inches='tight')
102 plt.show()
103 print("Dashboard saved as advanced_finops_dashboard.png")
104 return fig
105
106 # Run with existing data
107 dashboard = create_advanced_finops_dashboard(result, forecast, opt_result)

```

EXERCISE 8.5.4: Integrated Cost Dashboard



Dashboard saved as advanced_finops_dashboard.png

```

1 print("=" * 60)
2 print("CHALLENGE EXERCISE 8.5.5: Design Cost Optimization Strategy")
3 print("=" * 60)
4
5 challenge_scenario = {
6     "project": "Large Language Model Fine-tuning",
7     "requirements": {
8         "training_duration": 200,
9         "gpu_type": "A100",
10        "gpu_count": 8,
11        "precision": "FP32",
12        "instance_type": "on-demand",
13        "budget": 5000,
14    },
15    "constraints": {
16        "max_preemption_risk": "MEDIUM",
17        "min_accuracy": 0.95,
18        "deadline": "2 weeks",
19    }
20 }

```

```

21
22 print("CHALLENGE SCENARIO:")
23 print(f"    Project: {challenge_scenario['project']}")
24 print(f"    Baseline: {challenge_scenario['requirements']['gpu_count']}x "
25         f"{challenge_scenario['requirements']['gpu_type']} for "
26         f"{challenge_scenario['requirements']['training_duration']}h")
27 print(f"    Budget: ${challenge_scenario['requirements']['budget']}") # Fixed KeyError path
28 print(f"    Deadline: {challenge_scenario['constraints']['deadline']}")
29
30 # =====
31 # STEP 1: Calculate baseline cost
32 # =====
33 print("\n" + "="*60)
34 print("STEP 1: Baseline Cost")
35 print("="*60)
36
37 gpu_type = challenge_scenario['requirements']['gpu_type']
38 gpu_count = challenge_scenario['requirements']['gpu_count']
39 duration = challenge_scenario['requirements']['training_duration']
40 budget = challenge_scenario['requirements']['budget'] # Fixed KeyError path
41 price = GPU_PRICING[gpu_type]
42
43 baseline_cost = duration * price * gpu_count
44 print(f"    {gpu_count}x {gpu_type} @ ${price}/hr for {duration}h")
45 print(f"    Baseline cost: ${baseline_cost:.2f}")
46 print(f"    Budget:          ${budget:.2f}")
47 print(f"    Over budget by: ${baseline_cost - budget:.2f} -> NEEDS OPTIMIZATION!")
48
49 # =====
50 # STEP 2: Multi-GPU analysis
51 # =====
52 print("\n" + "="*60)
53 print("STEP 2: Multi-GPU Analysis")
54 print("="*60)
55
56 gpu_counts = [1, 2, 4, 8, 16]
57 multi_result = analyze_multi_gpu_cost(duration, gpu_type, gpu_counts)
58 multi_df = multi_result['table']
59
60 print(f"\n{'GPU#':<8}{'Time(h)':<10}{'Total Cost':<14}{'Efficiency%':<12}")
61 print("-" * 50)
62 for _, row in multi_df.iterrows():
63     print(f"{int(row['gpu_count']):<8}{row['time_hours']:<10.1f}${row['total_cost']:<13.2f}{row['parallel_efficiency']:<11.1f}")
64
65 optimal_gpus = int(multi_result['optimal_gpu_count'])
66 optimal_time = multi_result['optimal_time']
67 optimal_cost = multi_result['optimal_cost']
68 print(f"\n    Optimal GPU count: {optimal_gpus} (cost: ${optimal_cost:.2f}")
69 print(f"    Deadline check: {optimal_time:.1f}h needed -> {336/optimal_time:.1f}x faster than single GPU")
70 print(f"    Within 2 weeks ({336}h): {'YES' if optimal_time <= 336 else 'NO'}")
71
72 # =====
73 # STEP 3: Optimization strategies selection
74 # =====
75 print("\n" + "="*60)
76 print("STEP 3: Optimization Strategies")
77 print("="*60)
78
79 # Selected strategies for this project
80 strategies = [
81     # Quick wins (LOW effort, LOW risk)
82     {"name": "Mixed Precision (AMP)", "savings_pct": 0.25, "implementation_effort": "LOW", "risk_level": "LOW",
83     {"name": "Early Stopping", "savings_pct": 0.20, "implementation_effort": "MEDIUM", "risk_level": "LOW",
84     {"name": "Optimize Batch Size", "savings_pct": 0.10, "implementation_effort": "LOW", "risk_level": "LOW",
85     # Medium effort
86     {"name": "Spot Instances (Hybrid)", "savings_pct": 0.40, "implementation_effort": "MEDIUM", "risk_level": "MEDIUM",
87     {"name": "Gradient Accumulation", "savings_pct": 0.10, "implementation_effort": "MEDIUM", "risk_level": "LOW",
88     # High impact
89     {"name": "Multi-GPU (Optimal Config)", "savings_pct": 0.35, "implementation_effort": "HIGH", "risk_level": "LOW",
90 ]
91
92 # Calculate cumulative savings
93 cum_savings = 1.0
94 for s in strategies:
95     cum_savings *= (1 - s['savings_pct'])
96 total_savings_pct = (1 - cum_savings) * 100
97

```

```

98 print(f"\nSelected 6 strategies (ordered by priority):")
99 for i, s in enumerate(strategies, 1):
100     print(f"    {i}. {s['name']}: {s['savings_pct']*100:.0f}% savings | {s['implementation_effort']} effort | {s['risk_level']} risk")
101
102 print(f"\n    Combined savings potential: {total_savings_pct:.1f}%")
103
104 # =====
105 # STEP 4: Final cost forecast with uncertainty
106 # =====
107 print("\n" + "="*60)
108 print("STEP 4: Final Cost Forecast")
109 print("="*60)
110
111 base_cost = optimal_cost
112 contingency_pct = 15 # Lower contingency since using proven strategies
113
114 for s in strategies:
115     base_cost *= (1 - s['savings_pct'])
116
117 contingency = base_cost * (contingency_pct / 100)
118 final_expected = base_cost + contingency
119 final_upper = final_expected * 1.15 # 15% uncertainty band
120
121 print(f"    Starting from multi-GPU optimal: ${multi_result['optimal_cost']:.2f}")
122 print(f"    After all optimizations:")
123 for s in strategies:
124     print(f"        {s['name']}: -${multi_result['optimal_cost']*s['savings_pct']:.2f}")
125 print(f"    Base optimized cost:    ${base_cost:.2f}")
126 print(f"    Contingency ({contingency_pct}%): +${contingency:.2f}")
127 print(f"    Final expected cost:    ${final_expected:.2f}")
128 print(f"    Upper bound (95% CI):    ${final_upper:.2f}")
129
130 # =====
131 # STEP 5: Verification
132 # =====
133 print("\n" + "="*60)
134 print("STEP 5: Verification")
135 print("="*60)
136
137 print(f"\n{'Criterion':<30}{ 'Required':<15}{ 'Achieved':<15}{ 'Status':}")
138 print("-" * 70)
139 under_budget = final_expected <= budget
140 print(f"{'Budget':<30}${budget:<14.2f}${final_expected:<14.2f}{ 'PASS' if under_budget else 'FAIL'}")
141 print(f"{'Max Preemption Risk':<30}{challenge_scenario['constraints']['max_preemption_risk']:<15}{ 'MEDIUM':<15}{ 'PASS'}")
142 print(f"{'Deadline':<30}{challenge_scenario['constraints']['deadline']:<15}{f'optimal_time:.0f}h':<15}{ 'PASS'}")
143
144 # =====
145 # STEP 6: Strategy Summary
146 # =====
147 print("\n" + "="*60)
148 print("STEP 6: STRATEGY SUMMARY")
149 print("="*60)
150 print("""
151 RECOMMENDED OPTIMIZATION STRATEGY FOR LLM FINE-TUNING PROJECT
152 =====
153
154 PHASE 1 - Quick Wins (Week 1):
155     1. Mixed Precision (AMP)          -> 25% savings
156         - Near-zero accuracy loss, easy to implement
157         - Just add GradScaler() and autocast()
158     2. Early Stopping                  -> 20% savings
159         - Monitor validation loss, stop when plateau
160         - Combined with AMP: saves both time AND cost
161     3. Batch Size Optimization         -> 10% savings
162         - Find largest batch that fits GPU memory
163         - Reduces epochs needed
164
165 PHASE 2 - Medium Effort (Week 2):
166     4. Hybrid Spot + On-Demand        -> 40% savings
167         - Use on-demand for critical training phases
168         - Use spot for exploration/tuning
169         - MEDIUM risk: implement checkpointing
170     5. Gradient Accumulation           -> 10% savings
171         - Effective larger batch with limited memory
172         - Works synergistically with AMP
173
174 PHASE 3 - Infrastructure (Ongoing):
175     6. Optimal GPU Configuration     -> 35% savings

```

```

176     - Use multi-GPU analysis to pick cheapest config
177     - Avoid overprovisioning
178
179 TOTAL EXPECTED SAVINGS: {:.1f}%
180 -----
181 Baseline cost:      ${:.2f}
182 Optimized cost:    ${:.2f}
183 Savings:           ${:.2f}
184 Budget:            ${:.2f}
185 Under budget:      {}
186 """.format(
187     total_savings_pct,
188     multi_result['optimal_cost'],
189     final_expected,
190     multi_result['optimal_cost'] - final_expected,
191     budget,
192     "YES" if under_budget else "NO (slightly over - reduce contingency or negotiate budget)"
193 ))

```

CHALLENGE EXERCISE 8.5.5: Design Cost Optimization Strategy

CHALLENGE SCENARIO:

Project: Large Language Model Fine-tuning
 Baseline: 8x A100 for 200h
 Budget: \$5000
 Deadline: 2 weeks

STEP 1: Baseline Cost

8x A100 @ \$3.67/hr for 200h
 Baseline cost: \$5872.00
 Budget: \$5000.00
 Over budget by: \$872.00 -> NEEDS OPTIMIZATION!

STEP 2: Multi-GPU Analysis

GPU#	Time(h)	Total Cost	Efficiency%
1	200.0	\$734.00	100.0
2	111.1	\$815.56	90.0
4	58.8	\$863.53	85.0
8	30.8	\$903.38	81.2
16	18.9	\$1112.54	66.0

Optimal GPU count: 1 (cost: \$734.00)
 Deadline check: 200.0h needed -> 1.7x faster than single GPU
 Within 2 weeks (336h): YES